

Receiver-Guided Reinforcement Learning-based Congestion Control for IoT

Qihang Jiao, *Student Member, IEEE*, and Lotfi Mhamdi, *Senior Member, IEEE*

Abstract—The abstract goes here.

Index Terms—IoT, TCP, congestion control, reinforcement learning.

I. INTRODUCTION

THIS part is to be completed.

II. RELATED WORKS

III. SYSTEM DESIGN

Our proposal contains three modules in the receiver side for adjusting sender *cwnd*, Receiver Measurement Module, Window Decision Module, and Window Enforcement Module. Additionally, TCP sockets in both sender and receiver are extended or modified for adding interfaces to achieved our features.

A. Message types definition

The receiver TCP socket is extended to support communication with the RL agent. Three types of messages are defined:

- **State**: carries the observed network state and is sent to the agent for storage in the replay buffer.
- **Action Request**: triggers the agent to infer and return a control action based on the current state.
- **Connection Close**: notifies the agent that a flow has terminated, allowing it to clear the corresponding internal states.

B. Receiver Measurement Module

Our scheme relies on reliable state observations to ensure stable training. Therefore, the states in our design are required to reflect the congestion extent while remaining observable at the receiver. Despite RTT is a fine-grained and valuable indicator of congestion, it is difficult to directly observe in the receiver side.

To address this limitation, we derive a receiver-side congestion extent metric based on the *cwnd* elapsed time. We extend the sender TCP feature to include its *cwnd* in TCP header. In standard TCP congestion avoidance, the congestion window *cwnd* is increased by 1 MSS after an entire *cwnd* bytes has been acknowledged, resulting in an approximate growth rate

Qihang Jiao and Lotfi Mhamdi are with the School of Electronic and Electrical Engineering, University of Leeds, Leeds, UK (e-mail: el22qj@leeds.ac.uk; L.Mhamdi@leeds.ac.uk).

Lotfi Mhamdi is also with the College of Computing & Systems, Abdullah Al Salem University, Kuwait (e-mail: lotfi.mhamdi@asu.edu.kw).

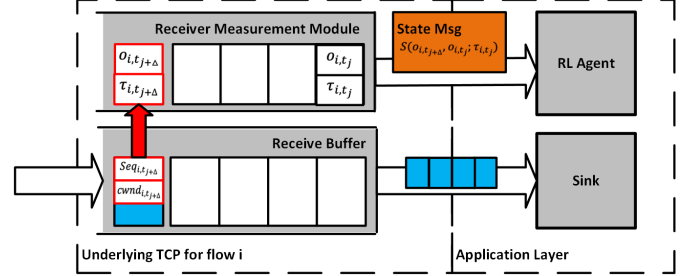


Fig. 1. An example of estimation task completion of flow i .

of 1 MSS per RTT, in which the method is named Additive Increment.

We utilise Additive Increment growing feature to enable the receiver to estimate network congestion extent by calculating the elapsed time for a flow to complete its *cwnd*. In particular, upon every packet receive event that includes a *cwnd* value different from the last value received, the Receiver Measurement Module establishes a new estimation task. A new task is initialised with the current observation (o) and a threshold (τ), e.g., for flow i started at time t_0 and estimated at time t_j , defined as below:

$$o_{i,t_j} \triangleq [B_i(t_j), L_i(t_j), t_j] \quad (1)$$

$$B_i(t_j) = \sum_{n=t_0}^{t_j} RxBytes_{i,n}$$

$$L_i(t_j) = \sum_{n=t_0}^{t_j} Losses_{i,n}$$

$$\tau_{i,t_j} = Seq_{i,t_j} + cwnd_{i,t_j} \quad (2)$$

An estimation task is completed at $t_{j+\Delta}$ when a coming packet is seen with a sequence number satisfying $Seq_{i,t_{j+\Delta}} > \tau_{i,t_j}$. We calculate this task duration as the *cwnd* elapsed time i.e., $t_{elapsed} = t_{j+\Delta} - t_j$. The $t_{elapsed}$ is then used to calculate the goodput (g_{i,t_j}) and the packet loss rate (ℓ_{i,t_j}) during the estimation task, where $g_{i,t_j} = \frac{B_i(t_{j+\Delta}) - B_i(t_j)}{t_{elapsed}}$ and $\ell_{i,t_j} = \frac{L_i(t_{j+\Delta}) - L_i(t_j)}{t_{elapsed}}$. We then form a state message according to $s_{i,t_j} = S(o_{i,t_{j+\Delta}}, o_{i,t_j}; \tau_{i,t_j})$, where the generation process is defined as follows:

$$S(o_{i,t_{j+\Delta}}, o_{i,t_j}; \tau_{i,t_j}) = [g_{i,t_j}, \ell_{i,t_j}, t_{elapsed}]$$

The Receiver Measurement and State Generation process can be seen in Algorithm 1.

Algorithm 1 Receiver Measurement and State Generation

Input: Flow Id i , Time t_j , Arrived Packet p_{i,t_j} , Flow Observation Queue q_i

Output: State message s_{i,t_j}

- 1: Extract Seq_{i,t_j} , $cwnd_{i,t_j}$, and $RxBytes_{i,t_j}$ from p_{i,t_j}
 - 2: $\tau_{i,t_j} = Seq_{i,t_j} + cwnd_{i,t_j}$
 - 3: Obtain $Losses_{i,t_j}$ from $TcpSocket_i$
 - 4: $B_i(t_j) = B_i(t_{j-1}) + RxBytes_{i,t_j}$
 - 5: $L_i(t_j) = L_i(t_{j-1}) + Losses_{i,t_j}$
 - 6: $o_{i,t_j} = [B_i(t_j), L_i(t_j), t_j]$
 - 7: Enqueue o_{i,t_j} and τ_{i,t_j} into q_i
 - 8: Obtain o_i^{head} and τ_i^{head} from the head of q_i
 - 9: **if** $Seq_{i,t_j} > \tau_i^{head}$ **then**
 - 10: Dequeue o_i^{head} and τ_i^{head} from q_i
 - 11: $s_{i,t_j} = S(o_{i,t_j}, o_i^{head})$
 - 12: Send s_{i,t_j} as State Message to Agent
 - 13: **end if**
-

C. Window Decision Module

The states input is triggered by each receive event that is not able to be known when it happens. Also for the purpose to avoid the agent decision process blocking the normal TCP behaviour, we set a window decision maintained in the Receiver socket and updated periodically. The window decision is a value within $[0, 1]$ and set in every ACK header to the sender.

D. Window Enforcement Module

The sender is modified to leverage the window decision as a factor to update its $cwnd$ as following:

$$cwnd \leftarrow cwnd \times Scaling_Factor^{Decision}$$

E. Enforcing Window Generation by RL Agent

IV. SIMULATION SETUP AND RESULTS

V. CONCLUSION

The conclusion goes here.

ACKNOWLEDGMENT

The authors would like to thank...

REFERENCES

- [1] H. Kopka and P. W. Daly, *A Guide to L^AT_EX*, 3rd ed. Harlow, England: Addison-Wesley, 1999.